INSTITUTO FEDERAL FARROUPILHA • CAMPUS PANAMBI
Tecnologia em Sistemas para Internet • Programação Web IV

Autenticação e autorização com JWT

Aluno: Fabiano Heusner | **Data:** 27/10/2026
Projeto: BairroPageable — sistema de imóveis

---

1. Autenticação e autorização

Autenticação verifica quem é o usuário. Neste sistema, o login confere email e senha e devolve um token JWT assinado.

Autorização verifica quais ações esse usuário pode executar. Por exemplo: um usuário USER autenticado pode alterar o status de um imóvel, mas recebe 403 ao tentar cadastrar um imóvel. Um ADMIN pode fazer as duas operações.

2. Stateful e stateless

Em uma aplicação stateful, o servidor mantém uma sessão e o cliente envia um identificador para que ela seja consultada. Em uma aplicação stateless, cada requisição fornece suas próprias credenciais e não depende de uma sessão de login armazenada no servidor.

Aqui, o cliente envia `Authorization: Bearer <token>` em cada chamada protegida. Spring Security usa `SessionCreationPolicy.STATELESS`. Ainda existem dados persistentes no banco: stateless não significa ausência de banco de dados. A validação também consulta o usuário atual para rejeitar tokens de usuários excluídos ou cujo perfil mudou.

3. Semântica e sintaxe de JWT

JWT significa JSON Web Token e transporta declarações (claims). No formato assinado compacto usado neste trabalho, sua sintaxe é:

```
Base64URL(header).Base64URL(payload).Base64URL(signature)
```

O header contém `alg: HS256` e `typ: JWT`. O payload contém `iss` (emissor), `sub` (ID do usuário), `iat` (instante de emissão), `exp` (expiração), `jti` (identificador do token) e a claim da aplicação `role` (`ADMIN` ou `USER`). Os instantes usam segundos desde a época Unix. A assinatura HMAC-SHA256 protege a integridade do header e do payload, usando a chave do servidor. O servidor verifica assinatura, algoritmo, emissor, expiração e usuário antes de autorizar a operação.

Base64URL é uma codificação, não criptografia. O conteúdo é legível: senhas nunca são incluídas no token. As senhas são armazenadas no banco com hash BCrypt e não aparecem nas respostas da API.

PARTE PRÁTICA

4. Implementação

Método e rota	Acesso	Token
POST `/auth/login`	Público	Não
GET `/imoveis`	Público	Não
GET `/imoveis/imoveis-page`	Público	Não
GET `/imoveis/{id}`	Público	Não
POST `/imoveis`	ADMIN	Sim
PUT `/imoveis/{id}`	ADMIN	Sim
DELETE `/imoveis/{id}`	ADMIN	Sim
PATCH `/imoveis/{id}/status`	ADMIN ou USER	Sim
GET/POST `/usuarios`	ADMIN	Sim
GET/PUT/DELETE `/usuarios/{id}`	ADMIN	Sim

Também são públicos a interface, a consulta de bairros, tipos e fotos e o formulário de interesse já existente (`POST /imoveis/sabermais/{id}`). Alterações de bairros, tipos e fotos exigem ADMIN. O endpoint legado de email não foi liberado: seu uso não é necessário para a atividade.

Uma rota protegida retorna **401 Unauthorized** quando falta o token ou ele é inválido/expirado. Um token válido de USER em uma operação exclusiva de ADMIN retorna **403 Forbidden**. A rota de status recebe um DTO específico, portanto não altera título, preço nem outros dados do imóvel.

Fluxo implementado

O cliente envia email e senha para `POST /auth/login`. Após conferir o hash da senha, o servidor emite um JWT. Nas rotas protegidas, o cliente envia o token no cabeçalho `Authorization: Bearer <token>`. O servidor valida o JWT e aplica a permissão correspondente à ação.

```
PATCH /imoveis/1/status
Authorization: Bearer <token-do-USER-ou-ADMIN>
Content-Type: application/json

{"status":"Reservado"}
```

VALIDAÇÃO E EXECUÇÃO

Evidências dos testes

A suíte Maven executou **16 testes, sem falhas ou erros**. Os cenários abrangem login incorreto, leitura pública, token ausente, inválido, expirado ou com assinatura adulterada, bloqueio de USER em rotas administrativas, gerenciamento de usuários e alteração exclusiva do status do imóvel. Os testes existentes de imóveis, tipos, fotos e email também passaram.

```
Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Os relatórios estão em `target/surefire-reports/` após a execução dos testes. O teste de segurança está em `src/test/java/com/example/demo/JwtAuthorizationApiTest.java`.

Como executar

No PowerShell, entre na pasta do projeto e execute:

```
$env:ADMIN_EMAIL = 'admin@example.com'
$env:ADMIN_PASSWORD = 'AdminTeste123!'
mvn '-Dmaven.repo.local=../.m2/repository' spring-boot:run
```

Acesse `http://localhost:8080/login.html`. O administrador inicial é criado somente se não houver usuários e as variáveis acima estiverem preenchidas. A senha apresentada é um exemplo para testar a atividade.

Requisitos: JDK 25 ou superior e Maven. Para repetir os testes:

```
mvn '-Dmaven.repo.local=../.m2/repository' test
```

Como demonstrar as permissões

Consulte imóveis sem login. Entre como ADMIN, cadastre tipo, bairro e imóvel e crie um usuário USER. Faça login com esse usuário e altere o status do imóvel. Uma tentativa de cadastrar imóvel com seu token recebe 403; a mesma tentativa sem token recebe 401. Na interface, essas respostas são apresentadas como mensagens; pela API, é possível conferir os códigos HTTP.

Observações da implementação

Sem configurar `JWT_SECRET`, a aplicação gera uma chave aleatória em memória, invalidando os tokens após reiniciar. A validade padrão é de uma hora. O logout remove o token da aba. A cópia de entrega começa com banco novo; os dados e fotos da pasta original não foram copiados.

Referência técnica

Spring Security. OAuth 2.0 Resource Server JWT.
docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html